

Database Transactions and ACID Properties

What a transaction is, the states it moves through, and the four guarantees every DBMS makes about it.

Agenda

1

What is a database transaction?

2

Transaction processing: commit vs. abort

3

Transaction states — the full state diagram

4

The four ACID properties

5

Bank transfer, traced: with and without atomicity

6

Transaction management in the DBMS

DEFINITION

What Is a Database Transaction?

A **transaction** is a logical unit of work — one or more read/write operations that must be treated as a single, indivisible action. Either every operation completes and is recorded permanently, or none of them are.

There is no valid outcome where the database ends up “half-updated.”

PKR 10,000 TRANSFER: A101 → A205

```
BEGIN TRANSACTION;
```

```
UPDATE Account  
SET balance = balance - 10000  
WHERE accountNo = 'A101';
```

```
UPDATE Account  
SET balance = balance + 10000  
WHERE accountNo = 'A205';
```

```
COMMIT;
```

Transaction Processing

Transaction processing guarantees that each transaction's effect on the database is exactly as if it had run alone — even with hundreds of others executing at the same instant, and even if hardware fails mid-transaction.

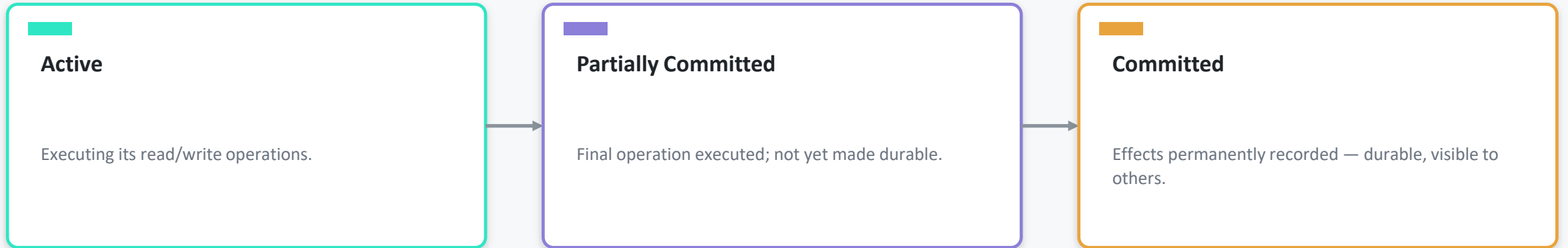
Commit

Every operation succeeded. The DBMS makes all of the transaction's changes permanent.

Abort (Rollback)

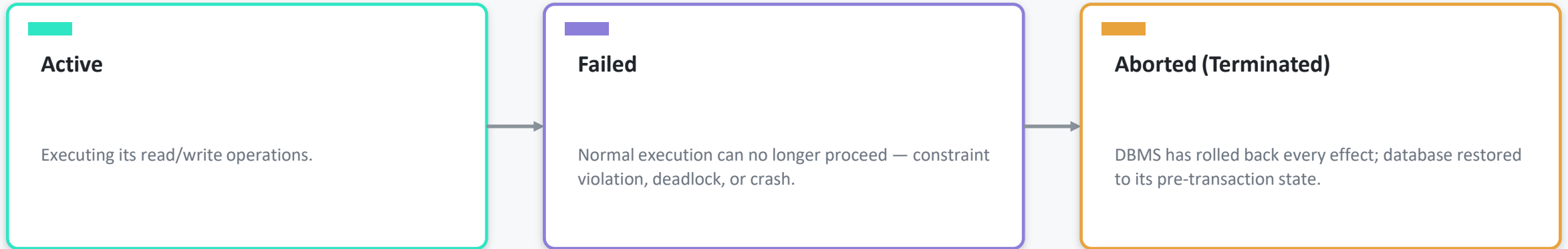
Something prevented completion. The DBMS undoes any partial changes, returning the database to its pre-transaction state.

Transaction States — The Successful Path



Committed and Aborted are always final. Once a transaction commits, nothing — not even a later crash — is allowed to undo it. That guarantee is Durability.

Transaction States — The Failure Path



Note: Partially Committed can also transition to Failed — if the DBMS discovers, while finalizing, that it cannot guarantee durability, it moves to Failed and then Aborted instead.

The ACID Properties

Atomicity

All of a transaction's operations happen, or none do — no partial effect is ever left behind.

Consistency

A transaction takes the database from one valid state to another, never violating an integrity constraint.

Isolation

Concurrent transactions don't see each other's incomplete, in-progress effects.

Durability

Once committed, a transaction's effects survive any later crash.

Atomicity and Consistency

Atomicity

The “all-or-nothing” property: a transaction's operations are treated as a single, indivisible unit. If any operation cannot complete, the DBMS undoes every operation that did already run — as if the transaction never started.

Consistency

A transaction, run to completion, takes the database from one valid state to another — never violating a domain, entity, referential, or general constraint. A DBMS enforces the constraints it knows about; encoding the business rule itself (e.g. “a transfer must debit exactly what it credits”) is the application's job.

Isolation and Durability

Isolation

Concurrently executing transactions do not interfere with one another — each behaves as though it were the only one running. Without isolation, one transaction could read another's half-finished work. This is a large enough topic to get its own full lecture next.

Durability

Once a transaction commits, its effects are permanent — they survive any subsequent failure. Delivered by writing a log of every change to stable storage before confirming the commit, so even a total loss of memory can be recovered from. The mechanics are Lecture 31's subject in full.

WORKED EXAMPLE

The Bank Transfer, Traced

Outcome 1 — both statements succeed

Step	Operation	State
1	BEGIN TRANSACTION	Active
2	Debit A101 by 10,000	Active
3	Credit A205 by 10,000	Active
4	Final operation done	Partially Committed
5	Log flushed to disk	Committed

Outcome 2 — crash after debit, before credit

Step	Operation	State
1	BEGIN TRANSACTION	Active
2	Debit A101 by 10,000	Active
3	(system crash)	Failed
4	Recovery manager undoes step 2	Aborted
5	A101 restored to original balance	(over)

What Atomicity Prevents

● Without Atomicity

Outcome 2 would leave A101 debited and A205 never credited — PKR 10,000 simply gone, with no record of where it went.

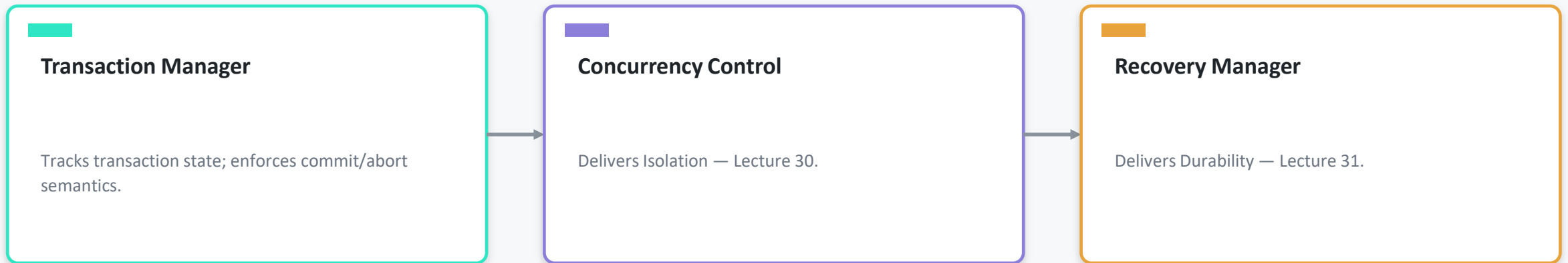
● With Atomicity

The recovery manager detects the incomplete transaction on restart, reverses the debit using the log — the books are exactly as if the transfer never happened.

Isolation without atomicity would still be broken: even perfect isolation between transactions says nothing about whether this transfer itself gets fully applied or fully reversed. All four ACID properties are independently necessary; none can substitute for another.

Transaction Management in the DBMS

The DBMS subsystem that tracks every transaction's state, enforces atomicity and consistency on commit or abort, and coordinates with two supporting subsystems.



Key Takeaways

- A transaction is a logical unit of work — one or more read/write operations that must succeed or fail as a whole.
- A transaction moves through well-defined states: Active, Partially Committed, and either Committed or (via Failed) Aborted.
- ACID — Atomicity, Consistency, Isolation, Durability — is the precise definition of a correctly executed transaction.
- The bank-transfer trace showed what atomicity prevents: without it, a crash mid-transfer silently destroys money.
- Transaction management coordinates concurrency control and the recovery manager to deliver the full ACID guarantee.